Lost & Found Uninorte

Modulo de Reclamacion y Verificacion de Objetos

Diseno de Software II — Patrones de Diseno

Marzo 2026

Integrantes del Equipo

#	Integrante	Patron Asignado
1	Integrante 1	**Abstract Factory** (Creacional)
2	Integrante 2	**Proxy** (Estructural)
3	Integrante 3	**Chain of Responsibility** (Comportamiento)
4	Integrante 4	**Visitor** (Comportamiento)

> *Reemplazar con los nombres reales del equipo*

Contexto del Proyecto

Que problema resolvemos?

En la Universidad del Norte se pierden objetos constantemente: laptops, audifonos, documentos, ropa...

Lost & Found Uninorte digitaliza el proceso de objetos perdidos:

- La administracion **registra** objetos encontrados con foto y categoria
- Los estudiantes **reclaman** objetos presentando **evidencias**
- Los administradores **verifican** la legitimidad de las reclamaciones
- Se genera **auditoria** y trazabilidad de todo el proceso

Arquitectura General

```
Frontend (React + Vite)
      |  HTTP REST
Backend (NestJS)
 |- Objects Module
 |- Claims Module
 |     |- Factories ......... Abstract Factory
 |     |- Proxy ............. Protection Proxy
 |     |- Handlers .......... Chain of Responsibility
 |     +- Visitors .......... Visitor
 +- Prisma ORM Service
      |
PostgreSQL (Docker)
```

Stack: NestJS + Prisma + PostgreSQL + Docker | React + Vite

Modelo de Datos (Prisma)

```
User ------< Claim >------ Object
               |
           Evidence[]
```

Modelo	Campos clave
User	id, email, name, role (ADMIN / STUDENT)
Object	id, description, photo, category, location, foundAt
Claim	id, status (PENDING/APPROVED/REJECTED), rejectionReason
Evidence	id, type, url, description

7 categorias: ELECTRONIC, COMMON, CLOTHING, STATIONERY, DOCUMENT, ACCESSORY, OTHER

Flujo Completo del Modulo

Paso	Accion	Patron
1	Estudiante crea reclamacion con evidencias	**Abstract Factory** valida evidencias segun categoria
2	Estudiante o admin consultan reclamaciones	**Proxy** filtra por rol y usuario
3	Admin verifica una reclamacion	**Chain of Responsibility** ejecuta validaciones en cadena
4	Admin audita una reclamacion	**Visitor** ejecuta operaciones sobre la estructura

Resultado del paso 3: APPROVED o REJECTED (con motivo y handler fallido)

Resultado del paso 4: Reporte de auditoria + puntuaciones de similitud textual

Patron 1

Abstract Factory

Patron Creacional

Integrante 1

Abstract Factory — Concepto General

> Provee una **interfaz para crear familias de objetos relacionados** sin especificar sus clases concretas.

Cuando se usa?

- Cuando un sistema debe ser independiente de como se crean sus productos
- Cuando hay **familias de objetos** que deben usarse juntos
- Cuando se quiere exponer solo **interfaces**, no implementaciones

Participantes

Rol	Descripcion
AbstractFactory	Declara operaciones para crear productos abstractos
ConcreteFactory	Implementa la creacion de productos concretos
AbstractProduct	Interfaz para un tipo de producto
Client	Usa solo las interfaces de la factory

Abstract Factory — Diagrama General

```
        +-------------------------+
        |    <<interface>>        |
        |    AbstractFactory      |
        |-------------------------|
        | + createProductA()      |
        | + createProductB()      |
        +------------+------------+
                     |
          +----------+----------+
          |                     |
+---------v--------+  +--------v---------+
| ConcreteFactory1 |  | ConcreteFactory2 |
|------------------|  |------------------|
| createProductA() |  | createProductA() |
| createProductB() |  | createProductB() |
+------------------+  +------------------+
          |                     |
          v                     v
  ProductA1 + B1         ProductA2 + B2
```

Ventaja clave: El cliente no sabe que clases concretas utiliza.

Abstract Factory — Nuestro Problema

Diferentes **categorias de objetos** requieren **diferentes tipos de evidencias**:

Categoria	Evidencias Requeridas
Electronicos (laptop, audifonos)	Numero de serie **O** factura digital
Comunes (ropa, documentos, utiles)	Descripcion detallada **Y** foto de referencia

Sin el patron: Un `if/else` gigante validando todo en el servicio.

Con el patron: Cada familia de objetos tiene su propia factory que sabe exactamente que evidencias validar.

Abstract Factory — Diagrama de Implementacion

```
          +---------------------------+
          |     <<interface>>         |
          |     ClaimFactory          |
          |---------------------------|
          | + validateEvidences()     |
          |   -> ValidationResult     |
          +-------------+-------------+
                        | implements
            +-----------+-----------+
            |                       |
+-----------v---------+  +----------v--------------+
| CommonClaimFactory  |  | ElectronicClaimFactory  |
|---------------------|  |-------------------------|
| validateEvidences() |  | validateEvidences()     |
|  DETAILED_DESC      |  |  SERIAL_NUMBER          |
|  REFERENCE_PHOTO    |  |  DIGITAL_INVOICE        |
+---------------------+  +-------------------------+
            ^                       ^
            +--- ClaimFactoryProvider ---+
                 getFactory(category)
```

Abstract Factory — Codigo: Interfaz

claim.factory.ts — El contrato abstracto

```
export interface ValidationResult {
  isValid: boolean;
  errors: string[];
}

export interface ClaimFactory {
  validateEvidences(evidences: EvidenceDto[]): ValidationResult;
}
```

Define **que deben hacer** las factories, no **como** lo hacen.

Abstract Factory — Factory Electronica

electronic-claim.factory.ts

```
@Injectable()
export class ElectronicClaimFactory implements ClaimFactory {
  validateEvidences(evidences: EvidenceDto[]): ValidationResult {
    const errors: string[] = [];

    const hasSerialOrInvoice = evidences.some(
      (e) => e.type === 'SERIAL_NUMBER' || e.type === 'DIGITAL_INVOICE'
    );

    if (!hasSerialOrInvoice) {
      errors.push(
        'Las reclamaciones electronicas requieren un '
        + '"Numero de Serie" o "Factura Digital".'
      );
    }

    return { isValid: errors.length === 0, errors };
  }
}
```

Abstract Factory — Factory Comun

common-claim.factory.ts

```
@Injectable()
export class CommonClaimFactory implements ClaimFactory {
  validateEvidences(evidences: EvidenceDto[]): ValidationResult {
    const errors: string[] = [];

    const hasDetailedDescription = evidences.some(
      (e) => e.type === 'DETAILED_DESCRIPTION' && e.description
    );
    const hasReferencePhoto = evidences.some(
      (e) => e.type === 'REFERENCE_PHOTO' && e.url
    );

    if (!hasDetailedDescription || !hasReferencePhoto) {
      errors.push(
        'Requieren "Descripcion Detallada" y "Foto de Referencia".'
      );
    }

    return { isValid: errors.length === 0, errors };
  }
}
```

Abstract Factory — Provider (Selector)

claim-factory.provider.ts

```
@Injectable()
export class ClaimFactoryProvider {
  constructor(
    private electronicFactory: ElectronicClaimFactory,
    private commonFactory: CommonClaimFactory,
  ) {}

  getFactory(category: ObjectCategory): ClaimFactory {
    switch (category) {
      case ObjectCategory.ELECTRONIC:
      case ObjectCategory.ACCESSORY:
        return this.electronicFactory;
      case ObjectCategory.COMMON:
      case ObjectCategory.CLOTHING:
      case ObjectCategory.STATIONERY:
      case ObjectCategory.DOCUMENT:
      case ObjectCategory.OTHER:
        return this.commonFactory;
      default:
        throw new BadRequestException(`No hay factory para: ${category}`);
    }
  }
}
```

Abstract Factory — Uso en el Servicio

claims.service.ts — Metodo create()

```
async create(createClaimDto: CreateClaimDto) {
  const { userId, objectId, objectCategory, evidences } = createClaimDto;

  // 1. Obtener la factory correcta segun categoria
  const factory = this.factoryProvider.getFactory(objectCategory);

  // 2. Validar evidencias con la factory especifica
  const validationResult = factory.validateEvidences(evidences);

  if (!validationResult.isValid) {
    throw new BadRequestException(
      `Evidencias invalidas: ${validationResult.errors.join(', ')}`
    );
  }
  // 3. Crear la reclamacion en BD...
}
```

El servicio **no sabe** que factory concreta se uso.

Abstract Factory — Beneficios

- **Extensibilidad:** Nueva categoria = nueva factory + registrar en provider
- **Open/Closed:** Las factories existentes no se modifican al agregar nuevas
- **Inyeccion de dependencias:** Las factories son `@Injectable()` de NestJS
- **Validacion desacoplada:** El servicio no conoce las reglas de cada categoria

> **Nueva categoria "MEDICAL"?**
>
> 1. Crear `MedicalClaimFactory implements ClaimFactory`
> 2. Agregar case en `ClaimFactoryProvider`
> 3. Listo. Sin tocar codigo existente.

Patron 2

Proxy

Patron Estructural

Integrante 2

Proxy — Concepto General

> Proporciona un **sustituto o representante** de otro objeto para **controlar el acceso** a el.

Tipos de Proxy

Tipo	Proposito
Protection Proxy	Controla acceso segun permisos — **el nuestro**
Virtual Proxy	Crea objetos costosos bajo demanda
Remote Proxy	Representa un objeto en otro espacio de direcciones
Logging Proxy	Registra las solicitudes al objeto real

Cuando se usa?

- Control de acceso mas sofisticado que una simple referencia
- Anadir funcionalidad transversal sin modificar el objeto real

Proxy — Diagrama General

```
                +---------------------+
                |   <<interface>>     |
                |   Subject           |
                |---------------------|
                | + request()         |
                +----------+----------+
                           |
            +-------------+-------------+
            |                           |
+----------v----------+    +-----------v---------+
|    RealSubject      |    |       Proxy         |
|---------------------|    |---------------------|
| + request()         |<---| - realSubject       |
|   (logica real)     |    | + request()         |
+---------------------+    |   checkAccess()     |
                            |   log()             |
                            +---------------------+
```

El cliente interactua con el Proxy creyendo que es el objeto real.

Proxy — Nuestro Problema

Las reclamaciones contienen **datos sensibles**: evidencias, informacion personal, fotos...

Quien	Que puede ver
STUDENT	Solo **sus propias** reclamaciones
ADMIN	**Todas** las reclamaciones
STUDENT	NO puede filtrar por estado ni por fechas
ADMIN	Filtros avanzados de busqueda

Sin el patron: Logica de autorizacion mezclada en cada metodo del servicio.

Con el patron: Un Proxy intercepta y controla el acceso.

Proxy — Diagrama de Implementacion

```
ClaimsController
      |
      |--- lecturas (GET) --------+---- escrituras (POST/PATCH/DELETE)
      |                           |
+-----v-------------------+      |
| ClaimsServiceProxy      |      |
|-------------------------|      |
| + findAll(context)      |      |
| + findOne(id, context)  |      |
| + findByStatus(ctx)     |      |
| + findByDateRange(ctx)  |      |
|   checkAccess()         |      |
|   logClaimAccess()      |      |
+-----------+-------------+      |
            | delega              |
+-----------v--------------------v---+
|        ClaimsService (real)         |
|-------------------------------------|
| findAll()  findOne()  findByStatus()|
| create()   update()   remove()      |
+-------------------------------------+
```

Proxy — Codigo: Contexto de Acceso

El Proxy necesita saber **quien** esta pidiendo los datos:

```
export interface ClaimAccessContext {
  userId: string;
  role: Role;   // ADMIN | STUDENT
}
```

Se extrae de los headers HTTP en el Controller:

```
private getContextFromRequest(request: Request): ClaimAccessContext {
  const role = request.headers['x-user-role'];
  const userId = request.headers['x-user-id'];
  // validaciones...
  return { role, userId };
}
```

Proxy — Codigo: Protection Proxy

claims.service.proxy.ts

```
@Injectable()
export class ClaimsServiceProxy {
  constructor(private readonly claimsService: ClaimsService) {}

  async findAll(context: ClaimAccessContext) {
    const claims = await this.claimsService.findAll();

    if (context.role === Role.ADMIN) {
      claims.forEach(claim => this.logClaimAccess(claim.id));
      return claims;    // Admin ve TODO
    }

    // Student solo ve SUS claims
    const ownClaims = claims.filter(c => c.userId === context.userId);
    ownClaims.forEach(claim => this.logClaimAccess(claim.id));
    return ownClaims;
  }

  async findOne(id: string, context: ClaimAccessContext) {
    const claim = await this.claimsService.findOne(id);
    if (context.role !== Role.ADMIN && claim.userId !== context.userId) {
      throw new ForbiddenException('Acceso denegado');
    }
    this.logClaimAccess(claim.id);
    return claim;
  }
}
```

Proxy — Restriccion y Logging

```
async findByStatus(status: ClaimStatus, context: ClaimAccessContext) {
  this.ensureAdmin(context);   // Barrera de acceso
  const claims = await this.claimsService.findByStatus(status);
  claims.forEach(claim => this.logClaimAccess(claim.id));
  return claims;
}

private ensureAdmin(context: ClaimAccessContext) {
  if (context.role !== Role.ADMIN) {
    throw new ForbiddenException('Acceso denegado');
  }
}

private logClaimAccess(claimId: string) {
  this.logger.log(`ACCESO A DATOS DE CLAIM: ${claimId}`);
}
```

Doble funcion del Proxy: Protection + Logging sin tocar el servicio real.

Proxy — Uso en el Controller

```
@Controller('claims')
export class ClaimsController {
  constructor(
    private readonly claimsService: ClaimsService,          // directo
    private readonly claimsServiceProxy: ClaimsServiceProxy, // proxy
  ) {}

  @Get()                // Proxy (lectura protegida)
  findAll(@Req() request: Request) {
    const context = this.getContextFromRequest(request);
    return this.claimsServiceProxy.findAll(context);
  }

  @Post()               // Directo (escritura)
  create(@Body() dto: CreateClaimDto) {
    return this.claimsService.create(dto);
  }
}
```

Proxy — Beneficios

- **Separacion de responsabilidades:** El servicio real NO conoce roles ni permisos
- **Single Responsibility:** Autorizacion aislada en su propia clase
- **Logging centralizado:** Todo acceso a datos sensibles queda registrado
- **Transparencia:** El Controller trabaja con la misma interfaz

> **STUDENT intenta filtrar por estado?**
> `ClaimsServiceProxy.findByStatus()` -> `ensureAdmin()` -> `ForbiddenException`
> El `ClaimsService` real **nunca se entero** del intento.

Patron 3

Chain of Responsibility

Patron de Comportamiento

Integrante 3

Chain of Responsibility — Concepto General

> Permite pasar solicitudes a lo largo de una **cadena de handlers**. Cada handler decide si procesa la solicitud o la pasa al siguiente.

Cuando se usa?

- Multiples **validaciones** que deben ejecutarse en secuencia
- El conjunto de handlers y su orden deben ser **configurables**
- Se quiere desacoplar el emisor de una solicitud de sus receptores

Analogia

> Triaje en urgencias: Enfermera -> Medico general -> Especialista -> Cirujano.
> Cada uno evalua. Si puede resolver, lo hace. Si no, lo pasa al siguiente.

Chain of Responsibility — Diagrama General

```
  Request
     |
     v
+---------+    +---------+    +---------+
|Handler 1|--->|Handler 2|--->|Handler 3|---> Exito
|---------|    |---------|    |---------|
|handle() |    |handle() |    |handle() |
+---------+    +---------+    +---------+
     |              |              |
     v              v              v
   Falla          Falla          Falla
```

Cada eslabon tiene dos caminos:

1. Pasa la validacion -> delega al siguiente handler
2. Falla -> lanza excepcion y rompe la cadena

Chain of Responsibility — Nuestro Problema

Cuando un **Admin verifica** una reclamacion, se ejecutan validaciones **en orden**:

Paso	Handler	Pregunta
1	**IdentityHandler**	El usuario existe y su rol es valido?
2	**AvailabilityHandler**	El objeto esta disponible (no reclamado)?
3	**EvidenceMatchHandler**	Tiene evidencia tipo serial?

Si todos pasan -> APPROVED.

Si cualquiera falla -> REJECTED con handler fallido y motivo.

Chain of Responsibility — Diagrama de Implementacion

```
              +-----------------------+
              |   BaseClaimHandler     |  (abstract)
              |-----------------------|
              | - nextHandler?         |
              | + setNext(handler)     |
              | + handle(context)      |
              +----------+-------------+
                         | extends
         +---------------+---------------+
         |               |               |
+--------v------+ +------v--------+ +----v--------------+
|IdentityHandler| |Availability   | |EvidenceMatch      |
|---------------| |Handler        | |Handler            |
|Usuario existe | |---------------| |-------------------|
|y rol valido?  | |Objeto no fue  | |Tiene evidencia    |
+---------------+ |reclamado?     | |SERIAL_NUMBER?     |
                  +---------------+ +-------------------+
```

Cadena: `identity.setNext(availability).setNext(evidenceMatch)`

Chain of Responsibility — Handler Base

base-claim.handler.ts

```
export abstract class BaseClaimHandler {
  private nextHandler?: BaseClaimHandler;

  setNext(handler: BaseClaimHandler): BaseClaimHandler {
    this.nextHandler = handler;
    return handler;  // permite encadenar: a.setNext(b).setNext(c)
  }

  async handle(context: ClaimVerificationContext): Promise<boolean> {
    if (!this.nextHandler) {
      return true;  // cadena termino exitosamente
    }
    return this.nextHandler.handle(context);
  }
}
```

Contexto que viaja por la cadena:

```
interface ClaimVerificationContext {
  claim: Claim & { user: User; object: Object; evidences: Evidence[] };
}
```

Chain of Responsibility — Handler de Identidad

identity.handler.ts — Eslabon 1

```
export class IdentityHandler extends BaseClaimHandler {
  constructor(private readonly prisma: PrismaService) { super(); }

  async handle(context: ClaimVerificationContext): Promise<boolean> {
    const user = await this.prisma.user.findUnique({
      where: { id: context.claim.userId },
    });

    if (!user) {
      throw new ClaimVerificationException(
        'IdentityHandler',
        'No existe un usuario activo asociado a la reclamacion.'
      );
    }

    if (user.role !== context.claim.user.role) {
      throw new ClaimVerificationException(
        'IdentityHandler',
        'El rol del usuario no es consistente.'
      );
    }

    return super.handle(context);  // Pasa al siguiente
  }
}
```

Chain of Responsibility — Handlers 2 y 3

availability.handler.ts — Eslabon 2

```
export class AvailabilityHandler extends BaseClaimHandler {
  async handle(context: ClaimVerificationContext): Promise<boolean> {
    const approvedByOther = await this.prisma.claim.findFirst({
      where: {
        objectId: context.claim.objectId,
        status: ClaimStatus.APPROVED,
        id: { not: context.claim.id },
        userId: { not: context.claim.userId },
      },
    });
    if (approvedByOther) {
      throw new ClaimVerificationException('AvailabilityHandler',
        'El objeto ya fue reclamado por otra persona.');
    }
    return super.handle(context);
  }
}
```

evidence-match.handler.ts — Eslabon 3

```
export class EvidenceMatchHandler extends BaseClaimHandler {
  async handle(context: ClaimVerificationContext): Promise<boolean> {
    const hasSerial = context.claim.evidences.some(
      e => e.type.trim().toUpperCase() === 'SERIAL_NUMBER');
```

Chain of Responsibility — Ensamblaje y Ejecucion

claims.controller.ts — POST /claims/:id/verify

```
@Post(':id/verify')
async verify(@Param('id') id: string) {
  // 1. Construir la cadena
  const identityHandler     = new IdentityHandler(this.prisma);
  const availabilityHandler = new AvailabilityHandler(this.prisma);
  const evidenceMatchHandler = new EvidenceMatchHandler();

  identityHandler.setNext(availabilityHandler).setNext(evidenceMatchHandler);

  try {
    await identityHandler.handle({ claim });
    // Todos pasaron -> APPROVED
    await this.prisma.claim.update({ data: { status: 'APPROVED' } });
  } catch (error) {
    // Algun eslabon fallo -> REJECTED
    await this.prisma.claim.update({
      data: { status: 'REJECTED', rejectionReason: error.reason }
    });
  }
}
```

Chain of Responsibility — Trazabilidad del Fallo

```
export class ClaimVerificationException extends Error {
  constructor(
    public readonly handler: string,  // Cual eslabon fallo
    public readonly reason: string,   // Por que fallo
  ) { super(reason); }
}
```

Respuesta HTTP cuando falla (409 Conflict):

```
{
  "message": "Reclamacion rechazada durante verificacion.",
  "eslabonFallido": "AvailabilityHandler",
  "motivo": "El objeto ya fue reclamado por otra persona.",
  "claimId": "abc-123",
  "status": "REJECTED"
}
```

Chain of Responsibility — Beneficios

- **Desacoplamiento:** Cada handler es independiente
- **Orden configurable:** Se puede cambiar orden o agregar pasos
- **Trazabilidad:** La excepcion indica el handler exacto que rechazo
- **Single Responsibility:** Cada handler valida exactamente una cosa

> **Nuevo requisito? Verificar que no sea fin de semana?**
>
> 1. Crear `WeekendHandler extends BaseClaimHandler`
> 2. Insertarlo en la cadena
> 3. Cero modificaciones a handlers existentes.

Patron 4

Visitor

Patron de Comportamiento

Integrante 4

Visitor — Concepto General

> Permite definir **nuevas operaciones** sobre elementos de una estructura **sin modificar las clases** de esos elementos.

Mecanismo clave: Double Dispatch

```
1. El cliente dice:     elemento.accept(visitor)
2. El elemento dice:    visitor.visitMe(this)
3. El visitor ejecuta:  su logica con los datos del elemento
```

Cuando se usa?

- Estructura de objetos **estable**
- Se necesitan **multiples operaciones** sobre esa estructura
- Se quiere evitar contaminar las clases con operaciones no relacionadas

Visitor — Diagrama General

```
+----------------+         +--------------------+
| <<interface>>  |         |  <<interface>>     |
| IVisitable     |         |  IVisitor          |
|----------------|         |--------------------|
| accept(v)      |         | visitElementA()    |
+-------+--------+         | visitElementB()    |
        |                  +---------+----------+
   +----+----+                 +----+-----+
   |         |                 |          |
 ElemA    ElemB            VisitorX   VisitorY
   |
   | accept(v) {           Cada Visitor tiene
   |   v.visitA(this)      su propia logica
   | }                     para cada elemento
   +-- Double Dispatch
```

Agregar `VisitorZ` **no requiere** modificar `ElemA` ni `ElemB`.

Visitor — Nuestro Problema

Sobre una reclamacion y sus evidencias necesitamos **operaciones diferentes**:

Operacion	Proposito
Auditoria	Reporte textual del claim y sus evidencias
Similitud textual	Comparar descripcion evidencia vs. objeto
(futuro)	PDF, riesgo de fraude, estadisticas...

Sin el patron: Metodos `generateAudit()`, `calculateSimilarity()` dentro de Claim y Evidence.

Con el patron: `ClaimElement` y `EvidenceElement` solo tienen `accept()`. Cada operacion vive en su propio Visitor.

Visitor — Diagrama de Implementacion

```
+----------------+            +---------------------+
| <<interface>>  |            |  <<interface>>      |
| IVisitable     |            |  IVisitor           |
|----------------|            |---------------------|
| accept(v)      |            | visitClaim(elem)    |
+-------+--------+            | visitEvidence(elem) |
        |                     +----------+----------+
   +----+----------+              +------+--------+
   |               |              |               |
ClaimElement  EvidenceElement  AuditVisitor  TextSimilarity
   |                              |            Visitor
   | accept(v) {             Genera reporte       |
   |   v.visitClaim(this)    de auditoria    Calcula Jaccard
   |   for each evidence:                   evidencia vs objeto
   |     evidence.accept(v)
   | }
```

Visitor — Codigo: Interfaces

visitor.interface.ts

```
export interface IVisitor {
  visitClaim(claimElement: ClaimElement): void;
  visitEvidence(evidenceElement: EvidenceElement): void;
}

export interface IVisitable {
  accept(visitor: IVisitor): void;
}
```

- **IVisitor** — define que tipos de elementos puede visitar
- **IVisitable** — define que un elemento puede ser visitado

Visitor — Elementos Visitables

claim.element.ts

```
export class ClaimElement implements IVisitable {
  constructor(public claim: ClaimWithRelations) {}

  accept(visitor: IVisitor): void {
    visitor.visitClaim(this);

    // Recorrido en cascada: visitar cada evidencia
    for (const evidence of this.claim.evidences) {
      const evidenceElement = new EvidenceElement(evidence, this.claim.object);
      evidenceElement.accept(visitor);
    }
  }
}

export class EvidenceElement implements IVisitable {
  constructor(
    public evidence: Evidence,
    public relatedObject: Object,
  ) {}

  accept(visitor: IVisitor): void {
    visitor.visitEvidence(this);
  }
}
```

Visitor — AuditVisitor

audit.visitor.ts

```
export class AuditVisitor implements IVisitor {
  private auditReport: string[] = [];

  visitClaim(claimElement: ClaimElement): void {
    const c = claimElement.claim;
    this.auditReport.push('--- INICIO AUDITORIA ---');
    this.auditReport.push(`ID Reclamacion: ${c.id}`);
    this.auditReport.push(`Estado: ${c.status}`);
    this.auditReport.push(`Objeto: ${c.objectId}`);
    this.auditReport.push(`Evidencias: ${c.evidences.length}`);
    this.auditReport.push(`Creado: ${c.createdAt.toISOString()}`);
  }

  visitEvidence(evidenceElement: EvidenceElement): void {
    const e = evidenceElement.evidence;
    this.auditReport.push(
      `  -> Evidencia [${e.type}]: ${e.description || 'Sin desc'}`
    );
    if (e.url) this.auditReport.push(`     URL: ${e.url}`);
  }

  getReport(): string {
    this.auditReport.push('--- FIN AUDITORIA ---');
    return this.auditReport.join('\n');
  }
}
```

Visitor — TextSimilarityVisitor

text-similarity.visitor.ts

```
export class TextSimilarityVisitor implements IVisitor {
  private similarityScores: Array<{
    evidenceId: string; type: string; score: number
  }> = [];

  visitClaim(_claimElement: ClaimElement): void { /* no-op */ }

  visitEvidence(evidenceElement: EvidenceElement): void {
    const evidenceDesc = evidenceElement.evidence.description;
    const objectDesc = evidenceElement.relatedObject.description;
    const score = this.calculateSimilarity(evidenceDesc, objectDesc);
    this.similarityScores.push({
      evidenceId: evidenceElement.evidence.id,
      type: evidenceElement.evidence.type, score });
  }

  // Coeficiente de Jaccard: |A interseccion B| / |A union B|
  private calculateSimilarity(text1: string, text2: string): number {
    const words1 = new Set(this.tokenize(text1));
    const words2 = new Set(this.tokenize(text2));
    const intersection = [...words1].filter(w => words2.has(w)).length;
    const union = new Set([...words1, ...words2]).size;
    return Math.round((intersection / union) * 10000) / 100;
  }

  getScores() { return this.similarityScores; }
}
```

Visitor — Uso en el Controller

GET /claims/:id/audit

```
@Get(':id/audit')
async audit(@Param('id') id: string) {
  const claim = await this.prisma.claim.findUnique({
    where: { id },
    include: { evidences: true, object: true },
  });

  const claimElement = new ClaimElement(claim);
  const auditVisitor = new AuditVisitor();
  const similarityVisitor = new TextSimilarityVisitor();

  claimElement.accept(auditVisitor);
  claimElement.accept(similarityVisitor);

  return {
    auditReport: auditVisitor.getReport(),
    similarityScores: similarityVisitor.getScores(),
  };
}
```

Visitor — Beneficios

- **Open/Closed:** Nuevo Visitor, sin tocar ClaimElement ni EvidenceElement
- **Separacion de concerns:** Cada Visitor encapsula una sola operacion
- **Recorrido automatico:** `ClaimElement.accept()` propaga a evidencias hijas
- **Composabilidad:** Multiples visitors sobre la misma estructura

> **Nuevo requisito? Generar reporte PDF?**
>
> 1. Crear `PdfReportVisitor implements IVisitor`
> 2. Implementar `visitClaim()` y `visitEvidence()`
> 3. `claimElement.accept(pdfVisitor)`
> 4. Sin modificar ninguna clase existente.

Resumen de Patrones

Resumen Comparativo

Aspecto	Abstract Factory	Proxy	Chain of Resp.	Visitor
Tipo	Creacional	Estructural	Comportamiento	Comportamiento
Endpoint	POST /claims	GET /claims/*	POST /:id/verify	GET /:id/audit
Problema	Validar evidencias	Control acceso	Verificacion	Operaciones
Archivos	4	1	5	5
SOLID	Open/Closed	Single Resp.	SRP + OCP	Open/Closed

Conclusiones

Cada patron resuelve un problema concreto del dominio:

1. **Abstract Factory** — Que evidencias son validas para este tipo de objeto?
2. **Proxy** — Este usuario tiene permiso de ver esta informacion?
3. **Chain of Responsibility** — Esta reclamacion pasa TODAS las validaciones?
4. **Visitor** — Que operaciones ejecutar sobre esta reclamacion?

Stack: NestJS + Prisma ORM + PostgreSQL + Docker | React + Vite

Preguntas?

Lost & Found Uninorte

Diseno de Software II — Universidad del Norte